HARDWARE ENTROPY

Randomness From Physical Noise

A ring-oscillator true random number generator on the Xilinx Artix-7

Muhammad Saad Bin Waqas · **Ghulam Ishaq Khan Institute**

`Nexys A7-100T · XC7A100T-1CSG324C · Vivado 2018.2`

Algorithms cannot originate randomness

- A pseudorandom generator is a deterministic state machine — same seed in, same sequence out, every time.
- So the security of every key, nonce and token reduces to one question: was the initial seed unpredictable?
- A processor has no free variables. Its state is always a function of its previous state and its inputs.
- To get a value that is not a function of anything the designer controls, you must stop computing and start measuring.

Pseudorandom

A formula. Reversible if the seed is known or guessable. No physical input.

True random

A measurement of a physical process. No generating function exists to recover.

What the board actually does

5

ring oscillators

`3 / 5 / 7 / 9 / 11 stages`

390.6 k

raw samples / s

`100 MHz ÷ 256`

0.500

target output bias

`guaranteed by de-biasing`

Free-running oscillators
built from the FPGA's own logic cells — no external parts, no noise diode, no PLL.

Sampled into metastability
so the flip-flop's decision is made by thermal noise, not by the design.

Cleaned and proven
de-biased on-chip, then streamed to a laptop and tested statistically.

From thermal noise to a logic decision

1 Thermal noise

Carriers in the transistors carry thermal energy at any $T > 0$. The result is a fluctuating noise voltage: $v_n = \sqrt{4 k_B T R \Delta f}$

2 Phase jitter

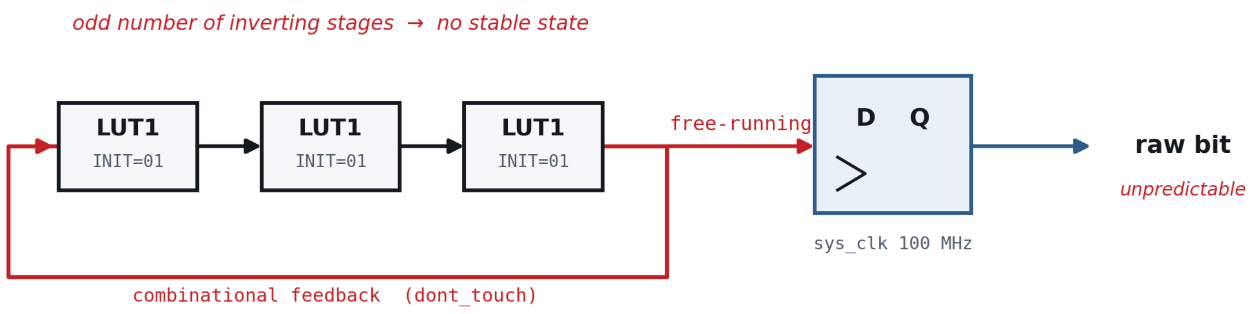
That noise shifts each switching threshold slightly, so every edge lands early or late. With no external reference to correct against, the error accumulates as a random walk in phase.

3 Metastability

When the sampler catches an edge inside its setup/hold window it sits at an unstable equilibrium. Which way it falls is decided by noise — a microvolt fluctuation amplified to a full logic level.

This is classical thermal noise amplified by an unstable equilibrium — not a quantum effect. Saying otherwise would misstate the physics.

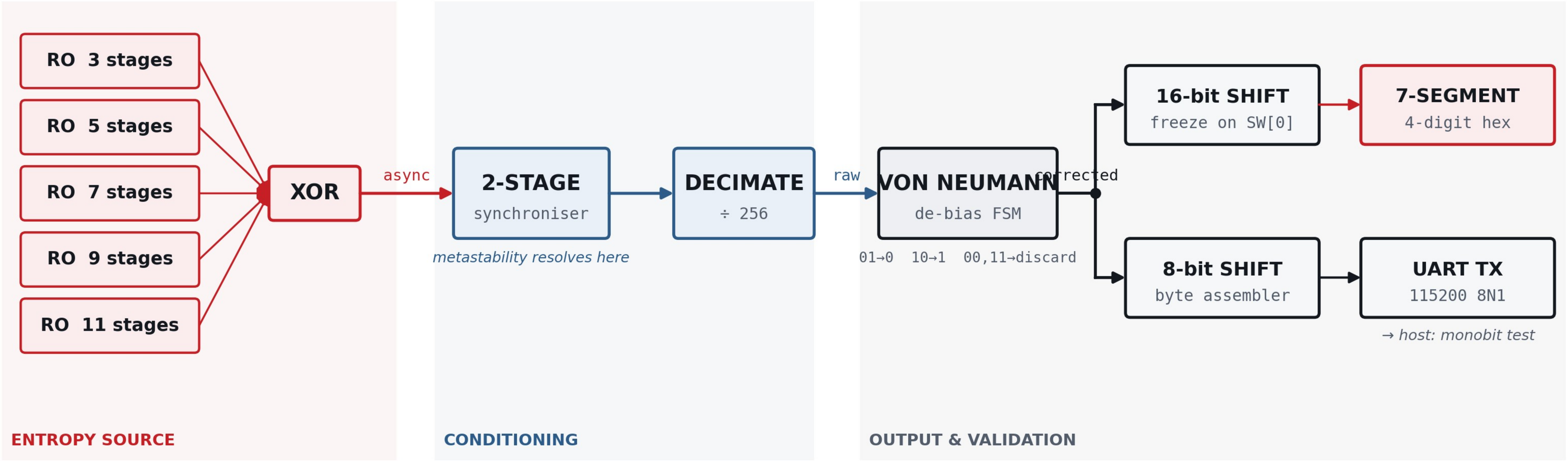
An odd ring has nowhere to settle



Written behaviourally, synthesis deletes this circuit. Each stage is instantiated as an explicit LUT1 primitive with dont_touch.

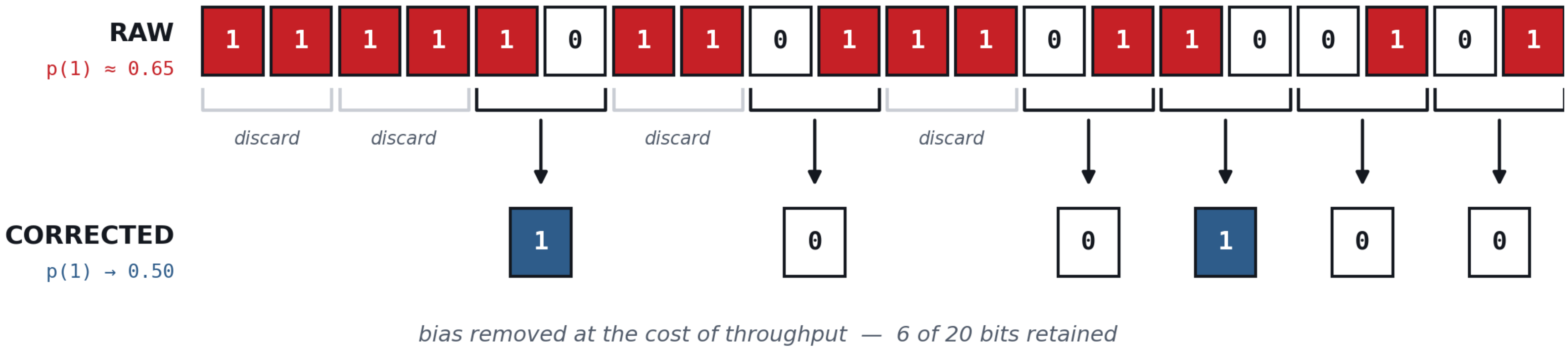
- Odd number of inverting stages → no consistent steady state → it oscillates on its own.
- Frequency set by its own propagation delay, which drifts with noise, temperature and supply.
- Five rings of different lengths run at different speeds, then XOR together.
- A flip-flop on the 100 MHz clock samples the combined edge.

The complete signal chain



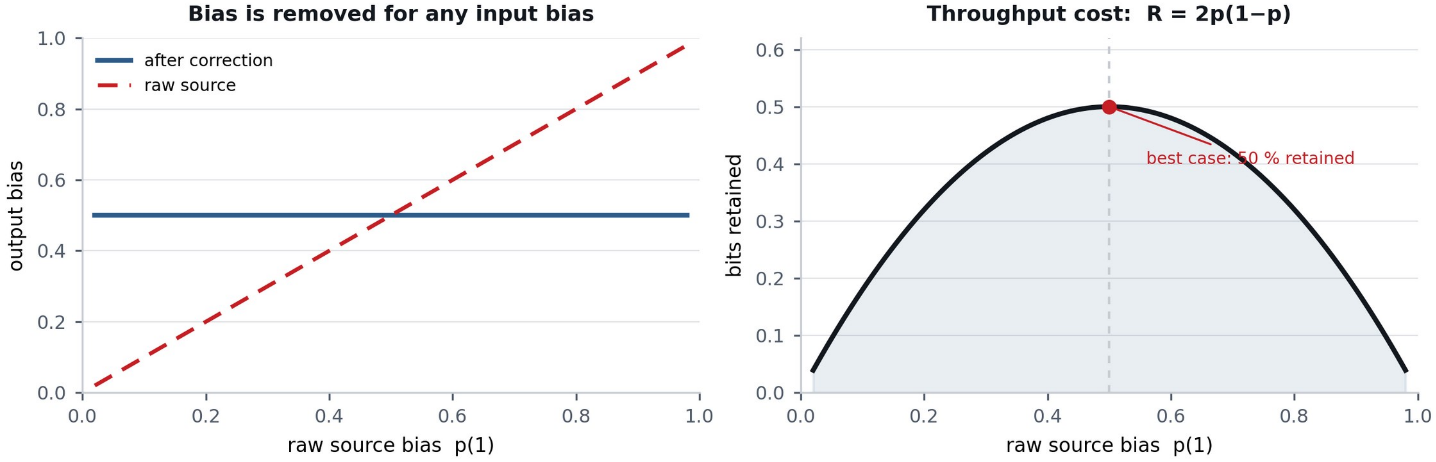
Entropy → conditioning → output. Every block runs on the FPGA fabric alone.

Removing the bias the physics leaves behind



Read in pairs. 01 → 0. 10 → 1. 00 and 11 are discarded. Because a source with bias p produces 01 and 10 with the same probability p(1−p), whatever survives is balanced exactly — without ever measuring the bias.

Uniformity is bought with throughput



The corrector fixes first-order bias only — it does not remove correlation between bits. That is why decimation happens upstream.

Guaranteed

Output bias lands on 0.5 for ANY input bias. Exact, not approximate.

Paid for

Retention $R = 2p(1-p)$. At best only half the bits survive; far fewer if the source is skewed.

Building the one circuit the tools exist to delete

1 — It gets optimised away

A behavioural inverter loop is collapsed to a single gate before implementation even starts.

Fix: instantiate each stage as an explicit LUT1 primitive carrying dont_touch.

2 — It fails the DRC

Static timing analysis needs a path with a start and an end. A loop has neither, so LUTLP-1 blocks bitstream generation outright.

Fix: ALLOW_COMBINATORIAL_LOOPS on the loop nets.

3 — Timing still complains

The DRC waiver alone is not enough; the analyser will still try to time the path.

Fix: set_false_path on the oscillator pins.

Both constraints are required — they address different stages.

Proving it, not asserting it

- Corrected bits are packed into bytes and streamed to a laptop over the board's USB-UART bridge at 115200 8N1.
- A short Python script counts ones and zeros and applies the NIST SP 800-22 monobit frequency test.
- The stream passes when the p-value is at least 0.01.

$$S = | n_1 - n_0 | / \sqrt{n}$$

$$p = \mathrm{erfc}(S / \sqrt{2})$$

One caveat, stated up front

The UART payload rate is lower than the corrected bit rate, so bytes are dropped when the transmitter is busy.

The captured file is therefore a subsample of the stream, not a continuous window. It is fine for a frequency test — but it should be said, not hidden.

Measured on the hardware

Quantity	Measured
Slice LUTs / registers	95 / 144
Bits captured	1,228,280
Ones / zeros observed	613,985 / 614,295
Measured p(1)	0.4999
Monobit p-value	0.7797
Verdict at $\alpha = 0.01$	PASS
Full NIST battery	7 / 15 passed

Confirmed by eye

Display blurs while running; freezing the switch holds one value, and each freeze gives a different number.

Why 8 of 15 fail

The 7 that pass are exactly the tests sensitive to bit balance. The 8 that fail hunt for periodicity and structure — which a ring oscillator, being a physically periodic circuit, genuinely has. Expected, not a flaw.

What this is not

Not certified

A deployable entropy source needs NIST SP 800-90B validation: an accredited lab, a documented noise model, and on-chip health tests.

None of that is claimed here.

One test is not a proof

The monobit test detects first-order bias only. It is blind to periodicity and to correlation between bits.

A deterministic alternating stream would pass it comfortably.

Shared environment

All five rings sit on one die and one supply rail. Different lengths decorrelate them, but they are not independent — temperature and voltage move them together.

What the board proves

The number on the display was not calculated. It was measured.

Thermal noise in the silicon, accumulated as oscillator phase jitter, resolved through metastability, de-biased on-chip and tested statistically — using nothing but the FPGA's own logic fabric.